# Talent Graph

A self-calibrating graph for discovering exceptional people

Working design note for a member network / talent discovery application

---

**Core idea.** Instead of attempting to define "talent" as a fixed objective number, treat it as a latent variable inferred from a graph of referrals, structured observations, pairwise comparisons, and longitudinal outcomes.

The system should answer three distinct questions:

1. How strong is our current evidence that a person is unusually capable?
2. How reliable is each member at identifying different kinds of capability?
3. How much of later success was discovered by the graph versus created by the opportunities the graph itself supplied?

The goal is not to construct a perfect ranking. The goal is to build a **self-correcting discovery mechanism** that becomes better at finding under-recognized talent as evidence accumulates.

## 1. The Graph

Let

$$G = (V, E)$$

where each node $v \in V$ represents a person, and each directed edge

$$e_{uv} : u \to v$$

represents person $u$'s referral or judgment of person $v$.

Each node carries several latent or learned values.

A useful representation is

$$\mathbf{z}_v = \begin{bmatrix} T_v \\ C_v \\ P_v \end{bmatrix}$$

where

- $T_v$: inferred underlying capability,

- $C_v$: realized contribution to the network,
- $P_v$: predictive or scouting ability.

In practice, these should eventually be vectors rather than scalars.

For example,

$$\mathbf{T}_v = \begin{bmatrix} t_{\text{problem}} \\ t_{\text{learning}} \\ t_{\text{agency}} \\ t_{\text{taste}} \\ t_{\text{output}} \\ t_{\text{generativity}} \\ t_{\text{originality}} \end{bmatrix}.$$

Similarly, a judge's predictive ability can be domain-specific:

$$\mathbf{P}_u = \begin{bmatrix} p_{\text{engineering}} \\ p_{\text{research}} \\ p_{\text{founder}} \\ p_{\text{design}} \\ p_{\text{leadership}} \end{bmatrix}.$$

The graph therefore learns not merely *who is strong*, but also *who is good at identifying what kind of strength*.

## 2. Priming: Turning "Vibes" Into Evidence

The system should not begin by asking:

> "Rate this person from 1–10."

Absolute numerical judgments are poorly calibrated across people.

Instead, begin with a high-recall prompt that captures intuition:

> **Who is one of the smartest, most unusually capable, or most under-recognized people you know whom we should meet?**

Useful variants include:

- Who is much better than their credentials suggest?
- Who do you call when you are stuck on a genuinely difficult problem?

- Who has made you update your model of what one person can accomplish?
- Who became effective in a new domain unusually quickly?
- Who consistently sees something important before everyone else?
- Who makes unusually strong people around them better?

The first answer is intentionally "vibes-based."

The system then converts the intuition into evidence by asking:

> **What did you personally observe that caused you to believe this?**

This produces a basic referral tuple:

$$e_{uv} = (x_{uv}, c_{uv}, r_{uv}, q_{uv})$$

where

- $x_{uv}$: referral strength / conviction,
- $c_{uv}$: confidence,
- $r_{uv}$: relationship depth,
- $q_{uv}$: quality of supporting evidence.

The system should preserve the original qualitative explanation rather than discarding it after creating numerical features.

## 3. A Simple Adaptable Rubric

A rubric should anchor judgments to observable behavior rather than prestige, employer, education, charisma, or social status.

Each dimension may be scored from 0 to 4.

| **Dimension** | What it attempts to measure |
|---|---|
| **Problem solving** | Ability to make progress on difficult, ambiguous, novel problems. |
| **Learning velocity** | Speed at which the person becomes effective in an unfamiliar domain. |
| **Agency** | Ability to convert ambiguity into action without requiring excessive structure. |
| **Taste** | Ability to identify worthwhile problems, approaches, people, or opportunities. |
| **Output** | Ability to turn capability into concrete artifacts, results, discoveries, or systems. |
| **Generativity** | Ability to increase the effectiveness, ambition, or insight of people around them. |
| **Originality** | Frequency of producing non-obvious ideas, frames, or solutions that others did not generate. |

### 3.1. Behavioral score anchors

Instead of allowing every evaluator to invent their own meaning of "3/4," use common anchors.

| Score | Interpretation |
|---|---|
| 0 | Strong evidence that the behavior is absent or consistently weak. |
| 1 | Roughly ordinary relative to the relevant comparison group. |
| 2 | Clearly above ordinary peers; repeated evidence exists. |
| 3 | Unusually strong; experienced observers notice the difference. |
| 4 | Extreme; repeatedly produces outcomes that surprise strong domain experts. |
| N/O | Not observed. No numerical penalty should be applied. |

Every rating should ideally contain

$$J_{uvk} = (s_{uvk}, c_{uvk}, e_{uvk})$$

where

- $s_{uvk}$ is the score for dimension $k$,
- $c_{uvk}$ is evaluator confidence,
- $e_{uvk}$ is written evidence.

For example:

> **Learning velocity: 4/4.**
> High confidence. Joined a compiler project with no previous compiler experience and within three weeks found a correctness bug that multiple experienced engineers had missed.

The evidence is more valuable than the number.

## 4. Pairwise Comparison Instead of Absolute Ranking

Humans are often better at answering

> "Who would you trust more with this kind of problem?"

than

> "How talented is this person from 1–10?"

Therefore the application should generate pairwise comparisons.

For people $a$ and $b$, a Bradley–Terry style model can infer latent strength:

$$P(a > b) = \sigma(\theta_a - \theta_b)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

The system can perform separate comparisons for different dimensions:

$$P(a > b \mid k) = \sigma(\theta_{a,k} - \theta_{b,k}).$$

Examples:

- Who would you trust with a highly ambiguous technical problem?
- Who learns unfamiliar material faster?
- Who has stronger taste in choosing problems?
- Who would you recruit first into a five-person team where failure is expensive?

The latent score is then estimated by the model rather than manually invented by the evaluator.

## 5. Initial Node Weight From Referrals

Suppose candidate $v$ receives referrals from several existing members.

Let $Top_5(v)$ be the five strongest usable referral signals.

A raw formulation is

$$W_v = c \sum_{u \in Top_5(v)} x_{uv} p_u.$$

However, an unnormalized sum causes five mediocre referrals to dominate one extremely informative referral.

A better form is

$$S_5(v) = \frac{\displaystyle\sum_{u \in Top_5(v)} x_{uv}\, p_u\, d_{uv}}{\displaystyle\sum_{u \in Top_5(v)} p_u\, d_{uv}}.$$

Here

- $x_{uv}$ is referral strength,
- $p_u$ is the calibrated reliability of the referrer,
- $d_{uv}$ is an independence / diversity discount.

The initial node estimate can then be

$$\boxed{W_v^{(0)} = (1 - c_v)\mu + c_v S_5(v)}$$

where

- $\mu$ is the system-wide prior,
- $c_v \in [0, 1]$ is evidence confidence.

A candidate with only one weak referral therefore remains close to the prior, while a candidate with several strong independent referrals moves farther away.

## 6. Independence and Clique Discounting

Five referrals from one tight social cluster do not contain the same information as five independent referrals.

Let

$$\rho_{ab}$$

represent historical correlation between judges $a$ and $b$.

A simple diversity discount might be

$$d_{uv} = \frac{1}{1 + \lambda \sum_{j \in R(v),\, j \neq u} \rho_{uj}}.$$

High overlap between referrers therefore reduces marginal weight.

In an implementation, correlation could be estimated using:

- historical similarity of ratings,
- shared employer,
- shared school or community,
- frequent co-referral,
- direct social proximity,
- repeated use of the same underlying evidence.

This is important because the system should reward **independent convergence**, not popularity within a clique.

## 7. Longitudinal Observation

After a fixed observation period, for example six months, the system records realized evidence.

Let

$$R_v$$

represent observed realized contribution.

A simple update is

$$\boxed{W_v^{(6)} = (1 - \beta) W_v^{(0)} + \beta R_v}$$

where $\beta$ determines how much longitudinal observation is trusted relative to the initial graph.

However, raw outcome should not be interpreted as raw talent.

The member's outcome may instead be modeled as

$$R_v = T_v + A_v + I_v + \epsilon_v$$

where

- $T_v$: latent underlying capability,
- $A_v$: opportunity or support provided by the network,
- $I_v$: interaction effects with collaborators,
- $\epsilon_v$: noise.

Thus a better learning target is residual performance:

$$\boxed{R_v^* = R_v - \mathbb{E}[R_v \mid O_v]}$$

where $O_v$ captures opportunities received.

Interpretation:

> How much did this person outperform or underperform what would have been expected given the resources and opportunities the system itself gave them?

## 8. Learning Who Is Good at Identifying Talent

A core feature of the graph is that the evaluator is also a node.

If $u$ strongly predicts that $v$ is exceptional, the later evidence about $v$ provides information about $u$'s scouting ability.

Let the prediction error be

$$E_{uv} = (x_{uv} - R_v^*)^2.$$

Maintain an exponentially weighted estimate of evaluator error:

$$\bar{E}_u^{(t+1)} = (1 - \eta)\bar{E}_u^{(t)} + \eta E_{uv}.$$

The evaluator's reliability may then be defined as

$$\boxed{p_u = e^{-\tau \bar{E}_u}}$$

for tuning parameter $\tau > 0$.

Repeatedly accurate evaluators therefore gain weight.

Repeatedly inaccurate evaluators lose weight.

## 9. Shrinkage: Preventing Instant "Oracles"

A judge who makes two correct predictions should not immediately receive huge influence.

Let $n_u$ be the number of sufficiently evaluated predictions made by judge $u$.

Use shrinkage:

$$\boxed{\hat{p}_u = \frac{n_u}{n_u + \lambda} p_u + \frac{\lambda}{n_u + \lambda} \mu_p}$$

where

- $p_u$ is estimated judge reliability,
- $\mu_p$ is the population prior for judge reliability,
- $\lambda$ controls how much evidence is required before the system trusts an individual estimate.

This prevents early luck from generating runaway influence.

## 10. Learning Judge Bias

Suppose evaluator $u$ systematically rates certain kinds of people higher than their later evidence suggests.

Represent an observation as

$$y_{uvk} = \theta_{v,k} + b_{u,k} + \epsilon_{uvk}.$$

Here

- $\theta_{v,k}$: latent capability of candidate $v$ on dimension $k$,
- $b_{u,k}$: systematic bias of evaluator $u$,
- $\epsilon_{uvk}$: noise.

The adjusted observation becomes

$$y^*_{uvk} = y_{uvk} - \hat{b}_{u,k}.$$

Importantly, "bias" here need not be interpreted morally.

It simply means:

> A systematic residual pattern in how a person's predictions differ from later observations.

Possible learned tendencies include:

- credential preference,
- charisma preference,
- familiarity preference,
- domain-specific overconfidence,
- tendency to overrate close collaborators,
- tendency to underrate unconventional backgrounds.

## 11. Aggregated Talent Estimate

Combining reliability, independence, and bias correction gives a general form:

$$\boxed{W_v = \frac{\sum_u r_u\, d_{uv}\, (y_{uv} - b_u)}{\sum_u r_u\, d_{uv}}}$$

where

- $r_u$: evaluator reliability,
- $d_{uv}$: independence weight,
- $b_u$: evaluator bias estimate,
- $y_{uv}$: evaluator assessment.

In a vector model:

$$\boxed{\mathbf{W}_v = \frac{\sum_u \mathbf{r}_u \odot d_{uv} \odot (\mathbf{y}_{uv} - \mathbf{b}_u)}{\sum_u \mathbf{r}_u \odot d_{uv}}}$$

where $\odot$ denotes element-wise multiplication.

The application should avoid collapsing this vector into one global number unless a specific downstream decision requires it.

## 12. Uncertainty Must Be First-Class

Every estimate should contain both a mean and uncertainty:

$$W_v = (\mu_v, \sigma_v).$$

For example,

$$8.1 \pm 0.3$$

means something very different from

$$8.1 \pm 2.4.$$

The second person is not necessarily weaker.

They are simply less understood.

This is particularly important for finding hidden talent.

A useful discovery queue may prioritize:

$$\boxed{\text{Discovery Priority}_v = \text{high upside} + \text{high uncertainty} + \text{low external recognition}}$$

rather than merely sorting people by current mean score.

## 13. Reward Information Gain, Not Obvious Predictions

A judge should receive little additional credit for predicting that an already universally recognized superstar is strong.

Instead, scouting value should depend on surprise.

Let

$$\hat{R}_v^{-u}$$

represent the system's expected outcome for $v$ before incorporating evaluator $u$'s judgment.

Then evaluator $u$'s informational contribution can be approximated by

$$IG_{uv} = \left| R_v^* - \hat{R}_v^{-u} \right| \times \text{Accuracy}(u, v).$$

The ideal scout is not merely someone who identifies strong people.

The ideal scout identifies strong people **before the rest of the graph knows they are strong**.

## 14. The Self-Fulfilling Problem

Once graph scores influence admission, introductions, funding, mentorship, or attention, the graph becomes an intervention system.

The loop is:

$$\text{belief about talent} \to \text{opportunity} \to \text{outcome} \to \text{updated belief.}$$

Without correction, this can become:

$$W_v \uparrow \Rightarrow O_v \uparrow \Rightarrow R_v \uparrow \Rightarrow W_v \uparrow .$$

The system may then mistake its own allocation decisions for evidence that its original ranking was correct.

Therefore the application should explicitly track:

$$O_v = \text{opportunity exposure vector.}$$

This might include:

- number of introductions,
- access to collaborators,
- funding or resources,
- mentorship hours,
- visibility,
- project opportunities.

Longitudinal talent updates should depend on opportunity-adjusted outcomes, not raw outcomes alone.

## 15. Exploration Versus Exploitation

If the system gives opportunities only to its highest-ranked members, it can never determine whether lower-ranked people were incorrectly assessed.

Therefore some opportunities should be allocated experimentally.

A simple policy is

$$P(\text{opportunity} \mid v) = (1 - \varepsilon) f(W_v) + \varepsilon U(v)$$

where

- $f(W_v)$ favors strong current estimates,
- $U(v)$ introduces controlled exploration,
- $\varepsilon$ is the exploration rate.

This randomness is not inefficiency.

It is necessary for identifying model error.

## 16. The Role of an LLM

An LLM should not be treated as the final judge.

Avoid:

$$\text{profile} \rightarrow \text{LLM} \rightarrow 8.71.$$

The numerical precision would be largely cosmetic.

Instead, use an LLM as a normalization and evidence-processing layer:

$$\boxed{\text{human observation} \rightarrow \text{structured evidence} \rightarrow \text{LLM normalization} \rightarrow \text{human comparison} \rightarrow \text{statistical inference}}$$

An LLM may help:

- convert prose into the standard rubric,
- distinguish first-hand observation from hearsay,
- extract evidence supporting each dimension,
- flag unsupported claims,
- detect duplicate underlying evidence,
- create anonymized evaluation packets,
- generate pairwise comparisons,
- summarize disagreement between evaluators,
- identify missing dimensions.

The raw evidence should always remain available for audit.

## 17. Operational Definition of "Cracked"

Instead of pretending that "crackedness" is objectively measurable, define it operationally.

One useful definition is:

$$\boxed{\text{Crackedness} = P(\text{substantially exceeds expectation} \mid \text{difficult open-ended work})}$$

This turns the score into a prediction that can later be tested.

For example, rather than displaying

> Alice = 8.7 / 10

the application could display

> Estimated 74% probability of belonging to the top decile of observed members on open-ended technical problem solving.
>
> Confidence: medium.
>
> Primary evidence: three independent first-hand observations.
>
> Largest uncertainty: leadership and generativity not yet observed.

## 18. A Minimal Application Data Model

A first version does not need an extremely complicated architecture.

### Person

```
Person {
    id
    name
    joined_at
    profile_metadata
    talent_vector
    uncertainty_vector
    contribution_vector
    scouting_vector
}
```

### Referral

```
Referral {
    referrer_id
```

```
    candidate_id
    conviction
    confidence
    relationship_depth
    evidence_text
    evidence_type
    created_at
}
```

### Evaluation

```
Evaluation {
    evaluator_id
    candidate_id
    dimension
    score
    confidence
    evidence_text
    observed_at
}
```

### Pairwise Comparison

```
Comparison {
    evaluator_id
    person_a
    person_b
    dimension
    winner
    confidence
    created_at
}
```

### Opportunity

```
Opportunity {
    person_id
    type
    magnitude
    source
    started_at
    ended_at
}
```

**Outcome**

```
Outcome {
    person_id
    dimension
    observed_value
    evidence
    evaluator_id
    observed_at
}
```

## 19. Minimal Product Loop

A first practical version can operate as follows.

**Step 1. Referral**

Every trusted member nominates one to three people using prompts such as:

> Who is unusually capable but insufficiently legible through conventional credentials?

**Step 2. Evidence capture**

The application asks what specific observed behavior produced the belief.

**Step 3. Initial graph estimate**

Candidate weight is calculated from up to five strong independent referrals:

$$W_v^{(0)} = (1 - c_v)\mu + c_v S_5(v).$$

**Step 4. Structured evaluation**

Judges evaluate only dimensions they have actually observed.

**Step 5. Pairwise comparisons**

The system occasionally asks evaluators to compare two people on a specific capability.

**Step 6. Admission / interaction**

Some people receive opportunities based on current evidence, while a small fraction are deliberately explored despite uncertainty.

**Step 7. Longitudinal evidence**

At three, six, and twelve months, collect concrete behavioral and outcome evidence.

**Step 8. Opportunity adjustment**

Estimate:

$$R_v^* = R_v - \mathbb{E}[R_v \mid O_v].$$

**Step 9. Back-propagation**

Update the reliability and bias estimates of evaluators whose predictions can now be tested.

**Step 10. Discovery propagation**

High-calibration members receive somewhat more influence in future referrals, subject to shrinkage and influence caps.

## 20. What the Application Should Show

A person page should probably not lead with one giant score.

A better interface might show:

- capability vector,
- uncertainty by dimension,
- strongest supporting evidence,
- number of independent referrals,
- graph distance / communities of referrers,
- observed contribution,
- opportunity exposure,
- surprising positive residuals,
- surprising negative residuals,
- unresolved disagreement between judges.

A candidate might appear as:

**Candidate X**

High confidence: Problem solving, learning velocity, agency.

Medium confidence: Originality.

Insufficient evidence: Leadership, generativity.

Three independent nominations from two unrelated communities.

Strongest evidence: Repeatedly became productive in unfamiliar technical systems within weeks.

Model note: Capability estimate is substantially higher than external prestige signal.

## 21. Important Safeguards

The system should obey several design constraints.

1. **Do not equate missing evidence with low ability.**
2. **Do not allow employer or school prestige to directly dominate talent scores.**
3. **Do not reveal existing scores before independent evaluations are submitted.**
4. **Do not allow any single evaluator unlimited influence.**
5. **Discount highly correlated evaluators.**
6. **Separate scouts from downstream outcome evaluators when possible.**
7. **Track opportunities created by the network.**
8. **Maintain an exploration pool of uncertain or model-disfavored candidates.**
9. **Retain qualitative evidence alongside inferred numbers.**
10. **Display uncertainty rather than fake precision.**
11. **Periodically evaluate whether model score actually predicts future behavior.**
12. **Reward informational surprise rather than prestigious referrals.**

## 22. The Quantity Worth Optimizing

The most interesting candidates are not necessarily the people with the highest current scores.

A talent discovery system should especially seek people for whom

$$\boxed{\text{Observed capability} - \text{Expected capability} \gg 0.}$$

These are people the current model systematically underestimates.

Likewise, the most valuable scouts may be those for whom

$$\boxed{\text{Predictive accuracy on hidden talent} \gg \text{population baseline.}}$$

This changes the purpose of the system.

It is no longer merely:

> Find the highest-status people.

It becomes:

> Find people who repeatedly create positive surprise, and learn which members can detect that surprise before everyone else.

## 23. The Full Feedback Loop

The complete mechanism can be summarized as:

$$\boxed{\text{Vibe} \rightarrow \text{Evidence} \rightarrow \text{Comparison} \rightarrow \text{Prediction} \rightarrow \text{Opportunity} \rightarrow \text{Outcome} \rightarrow \text{Calibration}}$$

At the graph level:

$$u \xrightarrow[x_{uv}]{p_u} v \rightarrow R_v^* \rightarrow p_u'$$

and recursively:

$$\text{people judge people} \rightarrow \text{judgments are evaluated} \rightarrow \text{judges become calibrated} \rightarrow \text{future judgments improve.}$$

The resulting graph is therefore simultaneously:

- a referral graph,
- a reputation graph,
- a prediction system,
- a longitudinal observation system,
- an exploration mechanism,
- and a model of who is good at discovering whom.

## 24. MVP Recommendation

For version one, resist the temptation to implement the full probabilistic model.

Start with:

1. seven behavioral dimensions,
2. evidence-backed 0–4 evaluations,
3. "Not observed" as a valid value,
4. up to five independent referrals,
5. simple referrer reliability,
6. pairwise comparisons,

7. six-month reassessment,
8. opportunity tracking,
9. uncertainty,
10. raw evidence retention.

A practical initial scoring rule is:

$$S_v = \frac{\sum_{u \in Top_5(v)} x_{uv} \hat{p}_u d_{uv}}{\sum_{u \in Top_5(v)} \hat{p}_u d_{uv}}$$

followed by

$$W_v^{(0)} = (1 - c_v)\mu + c_v S_v.$$

At six months:

$$R_v^* = R_v - \widehat{\mathbb{E}}[R_v \mid O_v],$$

$$W_v^{(6)} = (1 - \beta) W_v^{(0)} + \beta R_v^*,$$

and evaluator prediction errors update

$$E_{uv} = (x_{uv} - R_v^*)^2.$$

Everything more sophisticated can be learned once the graph contains enough observations to justify it.

## 25. Design Principle

The model must never become the ground truth it is attempting to predict.

If the graph chooses the winners, gives those winners the best opportunities, observes that they succeed, and then uses their success to validate its own ranking, it has learned very little.

Therefore the system should preserve:

$$\boxed{\text{raw evidence} + \text{dissent} + \text{uncertainty} + \text{exploration}}$$

alongside every inferred score.

The graph is useful precisely when it remains capable of discovering that its current model is wrong.

*The objective is not to assign humans a permanent number.*
*The objective is to build a network that becomes increasingly good at recognizing unexpected capability before conventional systems do.*